



Final Report

EI2525 Electric Power Engineering Project, 9 credits

Multicriteria Load Control

Software and Hardware Implementation of Study

Iraklis Bournazos

Dio Randa Damara

Hadrien Guillaud

Multicriteria Load Control

Software and Hardware Implementation of Study

IRAKLIS BOURNAZOS

DIO RANDA DAMARA

HADRIEN GUILLAUD

EI2525 Electric Power Engineering Project, 9 credits

Date: December 1, 2025

Supervisors: Nathaniel Taylor

Examiner: Lina Bertling Tjernberg

School of Electrical Engineering and Computer Science

KTH Royal Institute of Technology

Abstract

This project develops a smart control system for a domestic electric water heater under dynamic electricity prices and residential Photovoltaic (PV) generation. A linear programming model schedules heater operation over 24 hours to minimise grid cost, maximise PV self-consumption, and respect comfort and hardware limits by modelling the tank as a finite thermal storage with standing losses and a 1 kW power cap. A real-time supervisory controller runs every 10 seconds, detecting surplus PV from live export measurements and temporarily boosting heater power without exceeding physical constraints, while later reducing heating in expensive periods to preserve the daily energy budget. A prototype with an Arduino controller, triac power stage, and zero-crossing detector has been developed but still needs to be fully tested in the laboratory, to confirm utility of duty-cycle modulation. Simulations indicate reduced grid cost and higher PV utilisation while keeping the tank state of charge within its prescribed bounds at all times, thereby ensuring that hot-water availability for the user is not compromised.

Keywords: smart water heater, demand response, photovoltaic self-consumption, linear programming, real-time control, model adaptive control

Acknowledgements

We would like to express our sincere gratitude to our supervisor, Nathaniel Taylor, whose guidance, support, and vision for the development of an intelligent load-control device were central to the success of this project. His direction, the provision of smart-meter data, his assistance during the experimental procedures, as well as his generous supply of the necessary hardware components, were all invaluable. Without his expertise and the excellent collaboration and communication throughout the course of the project, this work would not have been possible. We are truly thankful for the opportunity to learn from him and for the exceptional experience this project offered us. We would also like to express our thanks to Lina Bertling Tjernberg for enabling our participation in this course and for providing the opportunity to undertake such an engaging and meaningful project.

List of Acronyms and Abbreviations

- AC** Alternating Current (pp. 18–20)
- API** Application Programming Interface (pp. 13–16)
- DC** Direct Current (p. 18)
- DSO** Distribution System Operator (pp. 4–5)
- HAN** Home Area Network (pp. 10–11)
- HTTP** Hypertext Transfer Protocol (p. 9)
- kW / kWh** kilowatt / kilowatt-hour (throughout, esp. pp. 3–7, 14–19)
- LP** Linear Programming (pp. 5–7)
- MAE** Mean Absolute Error (p. 12)
- MCU** Microcontroller Unit (pp. 8–9, 18–19)
- NTMM** NTMM data logger (pp. 11, 24)
- PV** Photovoltaic (pp. 1–4, 13–16)
- RMS** Root Mean Square (p. 9)
- SOC** State of Charge (pp. 6–7, 21–22)
- SMHI** Swedish Meteorological and Hydrological Institute (pp. 11–12)
- SEK** Swedish Krona (pp. 4, 15–16)

Contents

1	Introduction	1
2	Technical system description	1
3	Theory	3
3.1	Cut costs and use more self-produced solar energy	3
3.2	Electricity Tariffs and Spot Prices in Sweden	3
3.3	LP-based Optimization	5
3.4	Electronic theory	8
4	Method	9
4.1	Load consumption forecasting	9
4.2	Solar Forecasting	12
4.3	Hot-Water Usage Modeling	13
4.4	Day-Ahead Spot Price Methodology	14
4.5	Electrical Devices for Water Heater Control and Power Measurement	15
4.6	Water Heater Scheduling	17
4.7	Real-time control application	19
5	Application	20
5.1	Software Implementation Results	20
5.2	Day-ahead optimised heating plan	23
5.3	In-Laboratory Assembly and Testing	24
5.4	Electronic implementation and testing	27
6	Limitations of the Study	28
7	Conclusion	30
8	Reference List	30

1 Introduction

In 2025, the energy transition and decarbonization of electricity grids are accelerating, driven by ambitious climate goals and increasing volatility in energy prices. In Europe, the liberalization of electricity markets has led to the widespread adoption of dynamic pricing, with ever-finer temporal granularity: since October 2025, spot prices in wholesale markets have shifted to a quarter-hourly resolution in Sweden and several neighboring countries [1], more accurately reflecting supply and demand fluctuations. This shift, combined with the rise of capacity-based tariffs (which penalize monthly peak consumption), makes intelligent management of residential loads not only relevant but economically critical for households.

At the same time, solar self-consumption is becoming more widespread, fueled by falling photovoltaic panel costs and incentives to reduce grid dependence. However, compensation mechanisms for feeding excess solar energy back into the grid often remain unattractive or unstable, encouraging households to maximize on-site consumption rather than sell surplus energy at low prices. In this context, flexible loads, such as electric resistive water heaters with storage represent a major opportunity: their consumption can be shifted to periods of low cost or high solar production without compromising user comfort.

This project is part of the EI2525 – Electric Power Engineering Project course at KTH, which combines project management theory with the practical application of methods to real technical challenges proposed by industry or aca-

demic partners. The goal is to design a smart control device, capable of optimizing consumption of the household.

2 Technical system description

To address these challenges and opportunities, a smart control system for domestic water heaters is proposed. This system is designed to intelligently manage energy consumption by leveraging both predictive and real-time data, ensuring economic efficiency, user comfort, and optimal use of renewable resources. Below, the technical architecture and core functions of this system are described in detail.

The system is based on three main criteria to process optimization.

- **Minimizing solar exports:** By aligning consumption with peaks in local photovoltaic production, thus reducing non-profitable power feed into the grid.
- **Cost reduction:** By leveraging dynamic pricing to heat water during the cheapest time slots if solar generation is not sufficient and by accounting the grid tariff costs based on monthly peak demand in order to avoid load spikes that would increase demand related charges.
- **Maintaining comfort:** By ensuring hot water availability according to user needs while respecting technical constraints (maximum power, heating duration, etc.).

The proposed system takes the form of a plug-in control box, that is connected between the wall the integrating a hybrid approach that combines:

- **Long-term optimization** (next-day): Based on day-ahead spot prices and the latest solar forecasts, computing a 24-hour heating schedule.
- **Real-time control:** Monitoring live PV and smart meter data to capture unexpected solar surpluses, reduce exports/curtailment, and override parts of the schedule when reality deviates from forecasts.

This solution aims to validate, in practice, the integration of day-ahead optimization and real-time control with power electronics, communications, and simulation, while developing hands-on skills in hardware-software integration and smart energy system management.

Its core functions are designed to ensure reliable operation, economic savings, and maximized solar self-consumption, even in the face of variable electricity prices and unpredictable weather conditions.

- **Forecast Retrieval:** This function collects day-ahead predictions for both electricity spot prices and expected solar photovoltaic generation. An accurate forecast is essential to make proactive scheduling decisions and align water heating with periods of low grid cost or abundant solar output.
- **Demand Estimation:** By analyzing historical data and using user-defined profiles, this function predicts when and how much hot water will be needed to ensure that the system schedules enough heating to meet comfort requirements without unnecessary energy use.
- **State Estimation:** This function determines the current state-of-charge of the hot water tank. Relying on model estimates to inform the controller about available energy reserves ensures reliable supply.
- **Day-ahead Optimization:** This function calculates an energy-efficient heating schedule for the next day, taking into account the projected PV production, electricity prices, expected demand, and user constraints. The goal is to minimize cost while maximizing solar self-consumption.
- **Real-time Control:** This function allows the system to respond flexibly to forecast errors or unexpected changes to optimize the energy use and cost by continuously updating the schedule based on the real-time PV generation and actual tank conditions,
- **Actuation and Scheduling:** This function translates optimized schedules into actionable commands for the water heater, such as power set points signals to enable direct and automatic operation.
- **Safety, Fallback, and Error Handling:** This function imposes minimum state-of-charge and maximum power limits, and

triggers backup heating if needed to guaranty user comfort. It also manages malfunctions or missing data to keep the system running safely.

- **Logging, Comparison, and Visualization:** This function tracks actual and scheduled data throughout the operation. It produces logs and visualizations for performance analysis and system refinement to support technical evaluation and user feedback.

3 Theory

3.1 Cut costs and use more self-produced solar energy

The combination of flexible boiler control and photovoltaic (PV) generation can transform residential energy by aligning big consumption periods (when heating water or air for example) with solar production and dynamic electricity pricing. This approach maximizes the use of on-site renewable energy, reducing reliance on the grid and lowering electricity costs - especially in markets where prices fluctuate or peak tariffs apply (like in Sweden).

By shifting heating operations to periods of high solar output, households can minimize wasted PV energy and avoid selling excess power back to the grid at low compensation rates. This not only improves energy efficiency but also enhances grid stability by acting as a controllable load, helping balance supply and demand as renewable energy adoption grows.

For users, flexible boiler control ensures reliable hot water availability while adapting to real-time conditions, such as unexpected solar surpluses or grid outages. It also reduces carbon emissions by prioritizing clean, self-generated energy over grid electricity, which may still depend on fossil fuels. All of that without impacting comfort.

As energy systems evolve, this solution remains adaptable, supporting future innovations like vehicle-to-grid integration and aligning with policies that reward demand flexibility and renewable energy use. It's a practical way to optimize energy consumption, cut costs, and contribute to a more sustainable energy future.

3.2 Electricity Tariffs and Spot Prices in Sweden

The Swedish electricity system separates the cost of energy from the cost of using the electricity grid. Consumers purchase energy from competitive retailers based on Nord Pool day-ahead prices, which are published at a 15-minute resolution [1]. These spot-price contracts expose the consumer to the real-time variability of the wholesale market, where prices may become extremely low or even negative during periods of high renewable generation. In such situations, consuming electricity from the grid can be more economical than using locally generated solar energy.

In addition, until the end of 2025, Swedish households with small-scale PV installations benefit from a tax rebate of approximately

0.60 SEK/kWh for electricity exported to the grid. This microgeneration tax credit significantly increases the effective compensation for surplus solar energy and can make export economically attractive compared to self-consumption. However, the tax rebate is scheduled to be discontinued from 1 January 2026 [2], which will reduce the profitability of exporting PV energy and further strengthen the incentive for local self-consumption. In this context, control strategies that shift flexible loads, such as electric water heating, towards periods of local solar production become increasingly relevant.

In parallel, distribution system operators (DSOs) such as *Ellevio*, *Vattenfall Eldistribution*, and *E.ON Energidistribution* apply capacity-based tariffs (*effekttariff*) to encourage a smoother demand profile and reduce stress on the network. For residential customers, Ellevio calculates the monthly capacity charge using the average of the customer’s three highest hourly consumption peaks [3]:

$$P_{\text{cap,month}} = \frac{1}{3}(P_{\text{peak1}} + P_{\text{peak2}} + P_{\text{peak3}}) \quad (1)$$

This value (in kW) is multiplied by a fixed tariff (SEK/kW), resulting in a monthly capacity fee. In addition, Ellevio incentivizes shifting consumption to off-peak hours by applying a reduced capacity tariff between 22:00 and 06:00 [3]. Such mechanisms reward households that avoid simultaneous operation of high-power appliances (e.g., EV charging, oven, washing machine, heating boiler), which would otherwise create expensive peaks. Other DSOs such as Vattenfall Eldistribution and E.ON Energidistribution ap-

ply similar capacity-based tariff structures, with small variations in fixed charges, peak definitions, and night-time discounts. Despite these differences, the underlying principle is consistent across Sweden: households are economically incentivized to reduce short high-power peaks and to shift flexible loads toward periods of low network stress.

At the transmission level, Svenska kraftnät operates the national grid under a separate tariff structure. Transmission customers pay a capacity fee based on subscribed power, along with usage charges that reflect energy losses in the network. Although these tariffs apply primarily to large industrial and utility-scale grid users, they illustrate the broader principle that the Swedish grid is priced to encourage efficient use of transmission capacity and limit congestion [4].

For the controller developed in this project, these tariff characteristics are important for two reasons. First, quarter-hourly spot prices provide the short-term economic signal for optimizing the boiler schedule. Second, the existence of peak-demand-based network tariff highlights the importance of avoiding short, high-power peaks. By smoothing the boiler’s consumption profile and shifting its operation to periods of low or negative prices, the proposed method aligns household demand with both market conditions and the incentives imposed by the Swedish tariff system.

In addition to minimizing the energy cost based on quarter-hourly spot prices, it is also desirable to avoid unnecessary contributions to the household’s monthly peak demand, which deter-

mines the capacity-based tariff. For this reason, the scheduling model can be extended to account for a forecast of the total household load. Whenever the predicted aggregate demand is close to, or above, a predefined peak threshold, the boiler is prevented from starting new heating cycles and its operation is instead shifted with priority to night-time hours (e.g., 22:00–06:00), when both the network load and the capacity tariff are lower.

This strategy reduces the probability that the electric water heater will set a new monthly peak, thereby limiting the capacity charge, while still ensuring that the daily energy requirement for hot water is met.

3.3 LP-based Optimization

Optimizing the working schedule of electric water heaters is a well-studied issue in modern household and smart grid energy management. The primary objective is to minimize system costs—typically electricity expenses—while upholding comfort, safety, and efficient utilization of local energy assets such as solar PV generation. These requirements introduce multiple time-dependent and interrelated constraints, making the problem well-suited for mathematical programming techniques such as Linear Programming (LP).

LP is a tool for solving multi-period scheduling problems in which all decision variables are continuous and the system’s constraints and objectives are expressed as linear relationships. In water heater scheduling, the LP framework mod-

els the system across a defined time horizon, divided into intervals (such as 15-minute slots throughout the day). For each interval, continuous decision variables describe the amount of energy delivered to the water heater, how much is sourced from the grid or local solar, and the thermal state of the hot water tank. By formulating comfort, equipment, energy conservation, and tariff requirements as linear constraints, and defining the cost function in terms of electricity use, the LP approach enables efficient, practical, and scalable optimization of water heater operation in dynamic residential environments [5].

Let the schedule be divided into t time intervals (e.g., 15 minutes). For each interval t , define the decision variables:

- P_t : Heater power setpoint (kW)
- $E_{in,t}$: Heater energy input for the interval (kWh)
- $G_{grid,t}$: Energy supplied from the grid in interval t (kWh)
- $S_{sol,t}$: Energy supplied from solar in interval t (kWh)
- $E_{soc,t}$: State-of-charge of the tank (kWh)
- $S_{plus,t}$: Slack variable for unmet demand (kWh)

Then, the objective function is to minimize total electricity cost over the day while applying a high penalty for any unmet hot water demand:

$$\text{Minimize } \sum_{t=1}^N (\text{Price}_t \cdot G_{grid,t} + \lambda_{slack} \cdot S_{plus,t}) \quad (2)$$

Where Price_t is the spot electricity price for interval t , and grid energy is purchased at this price, the solar energy is assumed zero-cost, and λ_{slack} is a large, fixed penalty constant (ensures feasibility but discourages slacking).

To guarantee safe, efficient, and practically feasible operation, the scheduling optimization is bounded by a set of physically and economically justified constraints that preserve user comfort and protect the equipment. The applied constraints are:

- Energy Balance with Solar Excess Utilization

The total heater energy input in each interval is supplied by a combination of on-site solar and grid import. This respects the principle of energy conservation and priority use of renewables, as recommended for grid flexibility. Only the surplus solar energy, after other household loads are met and can be used for water heating. The heater input must be met by excess solar plus grid:

$$E_{\text{in},t} = S_{\text{sol},t} + G_{\text{grid},t} \quad (3)$$

Solar usage is limited by PV excess:

$$S_{\text{sol},t} \leq \max(0, \text{PV}_t^{\text{avail}} - \text{Load}_t^{\text{other}}) \quad (4)$$

$$0 \leq P_t \leq P^{\text{max}} \quad (5)$$

$$0 \leq E_{\text{in},t} \leq P^{\text{max}} \cdot dt_h \quad (6)$$

- Storage Tank State Equation

The optimization captures the storage flexibility and impact on user comfort by explicitly modeling the thermal state of charge of the tank, including retention and losses. The first order hold:

$$E_{\text{soc},t} = r \cdot E_{\text{soc},t-1} + \eta E_{\text{in},t} - E_{\text{dem},t} + S_{\text{plus},t} \quad (7)$$

where $r = (1 - \lambda_{\text{loss}} \cdot dt_h)$ is retention per time step, η is heater efficiency, and $E_{\text{dem},t}$ is actual hot water drawn off (from demand profile or metered values). The comfort limits are enforced by the explicit bounds:

$$E_{\text{min}} \leq E_{\text{soc},t} \leq E_{\text{max}} \quad (8)$$

- Grid and Terminal Constraints

Failsafe constraints for grid and terminal constraints are used to ensure that the limits of the grid and equipment are not exceeded each interval. Ensuring tank state stays within allowed bounds and returns to a safe or target value at the end of the day guarantees both safety and comfort. Penalizing unmet demand (with a high slack cost in the objective) is a common technique in practical scheduling.

$$E_{\text{soc},N} \geq E_0 \quad (9)$$

Tank returns to at least starting state-of-charge by the end of the optimization horizon.

- Soft Constraint Mechanism

To reflect tariffs with penalties (like

Swedish Ellevio's "effekttoppar"/peak charges), the use of soft constraints for grid import at critical hours aligns the LP with real-world tariff structures and incentive markets now appearing in European grid operator programs, the objective can also include:

$$\sum_{t \in P} \lambda_{\text{peak}} \cdot G_{\text{grid},t} \quad (10)$$

where $\mathcal{P}$ is the set of intervals flagged as system peaks and λ_{peak} is a price penalty. In practice, Swedish capacity tariffs are calculated from the three highest hourly demand peaks per month, which yields a non-linear, month-level charge. In this work, it is approximated by a linear per-interval penalty term, where the set $\mathcal{P}$ contains intervals that are likely to coincide with system or customer peaks, and λ_{peak} is tuned so that the implied annual cost is consistent with the underlying capacity charge. This soft constraint approach is standard in current flexible tariff programming and can be dynamically flagged based on load or system heuristics [6].

This model also includes logic to guarantee a fallback to a conservative schedule if an optimal solution is not found, ensuring the continuity of service and equipment safety for the operation. If the optimization does not yield a feasible solution (as "Optimal" status), a fallback schedule is applied. A slack variable is introduced into the tank state of charge equation, representing unmet hot water demand in each interval. The slack variable is assigned a large penalty cost in

the objective function, so the optimizer will only allow unmet demand if there is no other feasible way to meet all system constraints, such as energy limits, tank safety, or grid import restrictions.

$$\min \sum_{t=1}^N \text{Cost}_t + \lambda_{\text{slack}} \cdot \text{Slack}_t \quad (11)$$

subject to:

$$E_{\text{soc},t+1} = r \cdot E_{\text{soc},t} + \eta \cdot E_{\text{in},t} - E_{\text{dem},t} + \text{Slack}_t \quad (12)$$

where λ_{slack} is set very high relative to typical energy prices, ensuring slack is only used as a last resort.

By considering time-varying prices and solar forecast, LP can shift heating loads to periods with lower grid prices or higher renewable availability, supporting demand response and self-consumption strategies. The flexibility of LP allows for the integration of additional system components, physical models, and even user preferences or dynamic pricing signal. This approaches are globally optimal within the scope of their input parameters and constraints. They are regularly employed for residential demand-side management, microgrid scheduling, and smart appliance coordination [7].

3.4 Electronic theory

3.4.1 System Overview and Component Selection

The system consists of two main processing units. A Raspberry Pi 5 with 8GB of RAM is in charge of data acquisition, optimization, and high-level control. A microcontroller Arduino UNO (MCU) is responsible of precise timing, zero-crossing detection, and triac triggering.

The Raspberry Pi retrieves real-time electricity consumption/production data (*normally from the HAN port of the smart meter - here it's gonna be requests to a website*), day-ahead spot prices (from Nordpool), and weather forecasts (from SMHI [8]). It then computes an optimal heating schedule and sends phase-angle control commands to the MCU via USB serial communication. The MCU ensures accurate power modulation by triggering a triac at the correct moment in the AC cycle, using zero-crossing detection for synchronization.

3.4.2 Communication and Data Flow

The smart meter data are retrieved via HTTP requests (simulating HAN-port readings). Spot prices are fetched from Elpriset Just Nu API [9] (Nordpool day-ahead market) and weather forecasts are obtained from SMHI API [8] to predict next day consumption. The Raspberry Pi then processes this data, runs the optimization algorithm, and generates a duty-cycle, which is sent to the MCU via USB serial communication.

The communication between the Raspberry

Pi and MCU follows a simple JSON-based protocol: `{"duty": 0.9}`

3.4.3 Delay and power

The power delivered to the resistive heater is controlled by adjusting the triac's firing delay (δt) in each AC half-cycle. The relationship between duty-cycle (ρ) and power is simplified and assumed to be linear, as the main voltage is far from being a perfect sine wave:

$$\delta t = \frac{T}{2} \times (1 - \rho) \quad \text{where } T = 20 \text{ ms} \quad (13)$$

(0 ms = full power, 10 ms = off).

An optocoupler detects the AC zero-crossing and triggers an interrupt on the MCU. This ensures the triac is fired at the precise moment to achieve the desired delay. A hardware timer on the MCU ensures accurate triggering, avoiding software latency issues.

3.4.4 Safety

Since this device is handling high-voltage, it is preferable to have isolation between high- (AC side) and low-voltage (electronic components). Optocouplers and opto-triacs use an LED and variable optotransistor to ensure isolation.

Figure 1 is a visual schematic of the electronic architecture and the table 1 is summarizing the utility of each component.

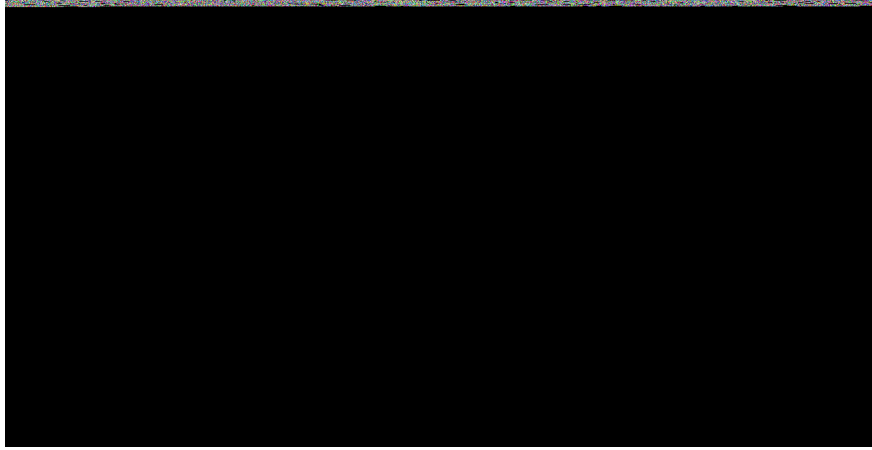


Figure 1: Electronic architecture: data acquisition, control, and actuation

Table 1: Key Electronic Components and Justification

Component	Selected Option	Justification
Main Controller	Raspberry Pi	Sufficient processing power for data acquisition, optimization, and HTTP requests. Supports Python for algorithm implementation.
Microcontroller (MCU)	Arduino UNO	Low-cost, real-time capable, with hardware timers for precise triac triggering.
Zero-Cross Detector	Optocoupler	Provides galvanic isolation between high-voltage AC and low-voltage MCU circuits.
Triac Driver	Optotriac	Ensures safe triggering of the triac while maintaining isolation.
Power Switch	Triac	Handles 220V AC loads, robust for resistive heating applications.
Current Sensor	Galvanically Isolated Current Sensor	Enables feedback on actual power consumption for closed-loop control.

4 Method

4.1 Load consumption forecasting

The load forecasting model presented in this section focuses on predicting the energy consumption of a freestanding 1950s–1970s house located in a Swedish town. The household, occupied by two adults and four teenagers or younger children, is characterized by a highly dynamic and variable electrical load profile. Heating is en-

tirely electric, relying on three air-air heat pumps supplemented by resistive heaters and occasional high-power devices such as a 2 kW radiant heater and a 2 kW portable fan heater. Water heating is provided by a 1 kW heater, with usage patterns varying daily based on occupant behavior [10].

The household’s modern appliances, including an induction cooker, a heat-pump dryer, and a washing machine, contribute significantly to

the load, alongside frequent use of smaller appliances such as a kettle, besides a coffee machines, toaster, etc. Background consumption includes multiple desktop computers, LED lighting, and plant lighting systems, while occasional tools such as a compressor, lawnmower, and chain-saw introduce short-term high-power spikes [10]. This diverse and fluctuating load profile makes accurate forecasting both challenging and essential for effective energy management.

Measurements of active and reactive power are obtained using a three-phase utility meter [10]. The meter provides two primary data streams: pulse-rate measurements from two LEDs, which flash proportionally to active and reactive energy consumption, and detailed 10-second interval data via a HAN-port using MBUS communication. The HAN-port data includes active and reactive power, voltage, and current per phase, enabling high-resolution monitoring of the household’s electrical consumption. To log these measurements, a Raspberry Pi is employed, using phototransistors to count LED pulses and an optocoupler to interface with the HAN-port. The raw data is decoded and saved in JSON and CSV formats for further analysis, ensuring a robust and continuous record of the household’s energy usage.

The forecasting methodology leverages historical consumption data combined with meteorological data provided by SMHI [8]. The goal is to predict the hourly consumption profile for the remainder of the current day and the following day. The process is divided into two stages: first, a `RandomForestRegressor` is trained to predict the average daily consumption based on

historical data and weather forecasts. This predicted average is then incorporated into a second model, a `MultiOutputRegressor` with a `RandomForestRegressor` base, which forecasts the hourly consumption profile. This two-step approach allows the model to capture the finer hourly variations in consumption.

The models are implemented in Python using libraries such as `pandas`, `scikit-learn`, and `seaborn` for data manipulation, model training, and visualization. The use of random forests provides robustness against overfitting and handles the non-linear relationships inherent in the data. The integration of meteorological data further enhances the model’s ability to account for weather-dependent consumption patterns, such as increased heating demand during cold periods. The performance of the forecasting models is evaluated using metrics such as Mean Absolute Error (MAE) and the coefficient of determination (R^2). A complete correlation matrix Figure 2 showing the relationships between the forecasting features and the predicted consumption is included in Appendix. Additionally, Figure 3 provided in the Appenix. compares the predicted and actual consumption values, providing a visual assessment of the model’s accuracy. While the models demonstrate strong predictive capability ($R^2 = 0.827$), uncertainties remain due to the stochastic nature of occupant behavior and the occasional use of high-power tools, which can introduce unexpected spikes in consumption (MAE = 412.25 W).

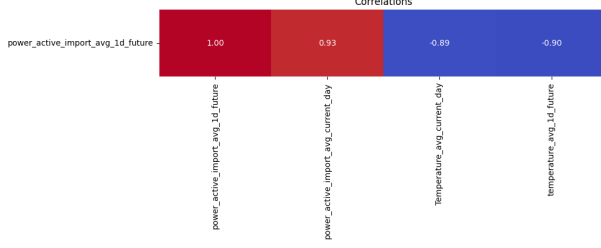


Figure 2: Correlation matrix of the mean active power consumption forecast algorithm

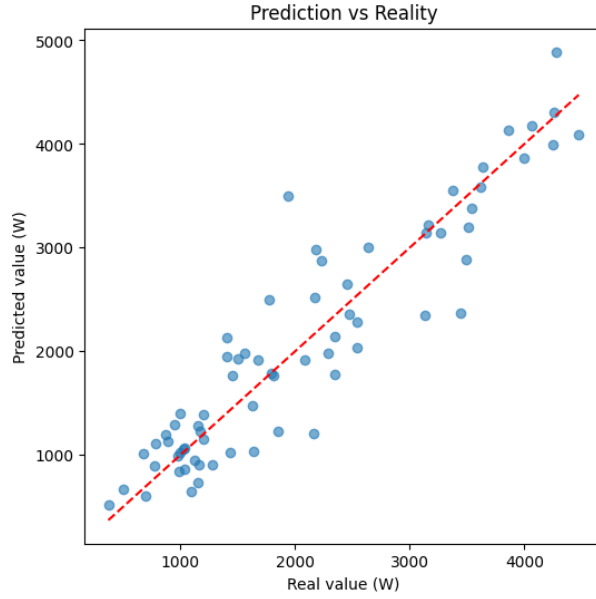


Figure 3: Comparison between predictions and actual average consumption values for the following day

4.1.1 Real-time forecasting pipeline

To integrate the forecasting models into the control architecture, a real-time prediction pipeline was implemented. The pipeline is executed automatically five minutes before every change of a day, producing an updated 24-hour load forecast at 15-minute resolution. This implementation retrieves live consumption data and mete-

orological forecasts, constructs the required feature vectors, and is using the two trained machine-learning models described earlier.

1. Data acquisition: Active power import measurements $P_{\text{in}}(t)$ are obtained from the NTMM data logger, which records samples at 10-second intervals. From these measurements, two aggregated quantities are computed:

$$\bar{P}_{\text{day}} = \frac{1}{N} \sum_{i=1}^N P_{\text{in}}^{(i)} \quad (14)$$

$$\bar{P}_{\text{hour}} = \frac{1}{M} \sum_{j=1}^M P_{\text{in}}^{(j)} \quad (15)$$

In parallel, hourly temperature forecasts are retrieved from the Open-Meteo API, yielding:

- today's mean temperature T_{day} ,
- tomorrow's mean temperature $T_{\text{day}+1}$,
- the next 24 hourly temperatures $\{T_{t+1}, T_{t+2}, \dots, T_{t+24}\}$.

2. Daily mean load forecast: The daily model $f_{\text{day}}(\cdot)$ predicts tomorrow's average power consumption as:

$$\hat{P}_{\text{mean}}^{(\text{tomorrow})} = f_{\text{day}}(\bar{P}_{\text{day}}, T_{\text{day}}, T_{\text{day}+1}) \quad (16)$$

3. Hourly load forecast: The predicted daily mean is then combined with real-time and meteorological features to form the input vector

of the hourly multi-output model:

$$\hat{P}_{t+h} = f_{\text{hour}}\left(\hat{P}_{\text{mean}}^{(\text{tomorrow})}, \bar{P}_{\text{hour}}, T_{\text{day}}, T_{t+1}, \dots, T_{t+24}\right) \quad (17)$$

Where $h = 1, \dots, 24$. After that the 24 hourly predictions, of the next day, are interpolated to 96 values at 15-minute intervals:

$$\hat{P}(t+k \cdot 15 \text{ min}) = \text{interp}\left(\hat{P}_{t+h}\right), \quad k = 1, \dots, 96 \quad (18)$$

The primary objective of the load forecasting model is to identify periods of potential excess energy relative to the household's photovoltaic (PV) production. With the excess solar production forecasts, one can maximize self-consumption of PV-generated electricity and reduces reliance on the grid. The model's ability to adapt to changing conditions, such as weather variations, ensures its relevance in real-world applications. By accepting short-term uncertainties as part of the optimization process, the model supports a flexible and adaptive energy management strategy, ultimately enhancing the efficiency and sustainability of the household's energy use.

4.2 Solar Forecasting

The provided PV installation that is used as a reference in this project consists of a small rooftop system with a total of ten (10) photovoltaic panels, each rated at 300 W. The panels are connected individually to microinverters, forming an AC-coupled architecture. The panels are arranged in distributed strings (4 + 3 +

2 + 1 modules) and it is assumed that they are facing south with a tilt angle close to 30°. The total nominal installed capacity of the system is therefore:

$$P_{\text{installed}} = 10 \times 300 \text{ W} \approx 3 \text{ kW}$$

Power and energy output from the installation are monitored on the AC side. Voltage is measured using a differential probe, and current is measured using a Pearson-101 current transformer. Measurements are sampled at 20 kHz using a Picoscope 3206B oscilloscope, averaged on-device, and transferred to a local server. The web interface provides real-time plots of $P(t)$ with second resolution and daily cumulative energy $E(t)$ for the investigated house installation. This real PV data is used exclusively for validation of the forecast model. The proposed day-ahead scheduling method does not assume that the real installation will always match the forecasted PV power. Instead, it applies a conservative forecast to avoid overestimation of solar availability.

More specifically, despite being subject to uncertainty, the PV forecast is not required to predict the exact production curve. Its role within the scheduling framework is to provide a strategic estimate of when a surplus of solar energy is likely to occur during the next day. Based on this estimate, the controller generates an initial plan for water-heater charging using the expected excess energy. The algorithm therefore accepts that the forecast may over- or underestimate the real production: if real PV output turns out lower than expected, the controller

performs an intra-day correction and shifts the remaining charging to later hours; if the real output is higher than expected, the boiler is simply stopped earlier.

To compute the forecast, a day-ahead estimation of PV power is obtained using the Forecast.Solar API [11]. The request is parameterized by the geographical location (Stockholm, Sweden), the nominal PV capacity (3 kWp), and the physical configuration of the system (panel azimuth 180° and tilt 30°). These parameters can be modified depending on the domestic installation being modeled. The API returns a predicted PV power time series covering the next 48 hours.

The retrieved forecasts undergo several processing steps in order to be transformed into usable data for the scheduling model. First, timestamps provided in UTC are converted to the local timezone (Europe/Stockholm) to ensure alignment with the electricity price tariff and the load-control logic. Only the forecast corresponding to the next calendar day is retained. The daylight interval (dawn–dusk) is then computed using the Astral library, and any predicted PV values outside this interval are forced to zero, ensuring that zero generation is enforced when PV output is physically impossible.

Because the scheduling optimization operates with 15-minute time steps, the forecast is resampled to a 15-minute resolution. It is noted that the present work uses the public version of the Forecast.Solar API, which provides hourly PV production forecasts without requiring an API key. For integration into the scheduling model, the hourly values are linearly

resampled to 15-minute resolution. In a future deployment or commercial application, a higher-resolution subscription (15-minute native forecasting) could be used to remove the interpolation step and further improve forecasting results.

Finally, each power forecast value is converted into energy per control interval (15 minutes). Since the forecast consists of a time series, the conversion is performed element-wise for every timestamp:

$$E_{\text{PV},15\text{min}}(t) = \frac{P_{\text{solar}}(t)}{1000} \cdot 0.25 \quad (19)$$

where $P_{\text{solar}}(t)$ is the forecasted PV power (W) at time t , and 0.25 h corresponds to the 15-minute interval. The result is a 15-minute energy profile (kWh), which is used as input to the optimization model.

4.3 Hot-Water Usage Modeling

Domestic hot-water consumption is measured using a pulse-output flow sensor installed on the hot-water piping of the reference installation. Each emitted pulse corresponds to a fixed water volume. According to the sensor specifications, one pulse represents:

$$V_{\text{pulse}} = 0.002 \text{ L/pulse}$$

The raw dataset contains irregularly sampled pulse events, stored as timestamps in Unix epoch format together with the cumulative pulse counter. Since pulses occur only when water is being consumed, periods of zero flow do not

generate data. After loading the dataset, timestamps are converted into localized datetime values, the cumulative pulse counter is differentiated and negative differences (caused by occasional counter resets) are clipped to zero, ensuring that only positive increments contribute to water consumption. This yields the incremental water volume for each sample:

$$\Delta L_i = \max(0, (\text{pulses}_i - \text{pulses}_{i-1}) \cdot V_{\text{pulse}}) \quad (20)$$

Because the energy-scheduling model operates in 15-minute resolution in the optimization, the water consumption signal is resampled using a sum aggregation:

$$L(t) = \sum_{i \in \text{interval } t} \Delta L_i \quad [\text{L per 15 min}] \quad (21)$$

To extract a representative daily usage pattern, each timestep is assigned a time of day index. All samples with the same index across the entire dataset for a whole year are grouped, and their average is computed:

$$\bar{L}(k) = \frac{1}{N_k} \sum_{d=1}^{N_k} L_d(k) \quad (22)$$

where $k \in \{1, \dots, 96\}$ corresponds to the 96 fifteen-minute intervals of a day. This averaging procedure produces a time-of-day consumption profile, which reflects the most likely hot-water usage pattern without depending on a specific day.

The water volume is then translated into energy demand using the standard physical heat model. The system assumes an inlet cold-

water temperature of approximately 10°C (typical value in Nordic climates) and a boiler target temperature of 60°C. The latter is selected because domestic water storage below 55°C can allow the proliferation of *Legionella* bacteria, and most international guidelines recommend 60°C for hygienic storage. Thus, the temperature rise is:

$$\Delta T = 60^\circ\text{C} - 10^\circ\text{C} = 50^\circ\text{C}$$

The thermal energy required to heat the volume in each 15-minute interval is:

$$E(t) = L(t) \cdot \rho \cdot c_p \cdot \Delta T \quad (23)$$

Using $\rho = 1 \text{ kg/L}$ and $c_p = 4.186 \text{ kJ}/(\text{kg} \cdot ^\circ\text{C})$, the expression becomes:

$$E(t) = L(t) \cdot 4.186 \cdot 50/3600$$

$$E(t) \approx 0.0581 \cdot L(t) \quad [\text{kWh per 15 min}]$$

The resulting time-series $E(t)$ represents the expected thermal demand of the boiler during each 15-minute interval and is used as a deterministic input to the day-ahead optimization model that coordinates solar production forecasts and electricity price signals.

4.4 Day-Ahead Spot Price Methodology

The day-ahead electricity price signal constitutes the main economic driver of the control strategy. Prices are retrieved automatically from the public API of elprisetjustnu.se [9], which provides Nord Pool day-ahead quarterly prices for each

Swedish bidding zone. The SE3 zone is selected, corresponding to the location of the installation in the Stockholm region. The fetched data include timestamps and corresponding prices in SEK/kWh.

Statistical checks are applied to ensure data consistency and to identify potential anomalies or opportunities in the control strategy, such as missing timestamps or negative prices. The latter may occur in the Nordic market during periods of oversupply or high renewable energy generation, indicating situations where consumers are effectively rewarded for consuming energy. Such instances are flagged within the control framework, as they represent opportunities for cost-free or even profitable operation of the electric boiler.

In practical terms, the controller distinguishes between periods of high and low market prices: when spot prices are normal or high, it prioritizes self-consumption of locally generated PV energy to minimize the purchase of expensive electricity. Conversely, when the day-ahead prices are very low or even negative, the algorithm temporarily shifts the priority toward grid consumption—fully exploiting these market opportunities by charging the boiler directly from the network instead of using solar generation. This logic acts as an initial economic filter that determines whether solar energy or grid power should be preferred for water heating.

The processed price vector

$$p(t) = [p_1, p_2, \dots, p_{96}] \quad (24)$$

is thus used as a time-indexed economic input to the optimization layer of the scheduling model. During normal operation, high price periods are associated with load curtailment, while low or negative price periods trigger the activation of flexible loads such as the water heater. This approach allows the controller to dynamically align the household’s energy consumption with market conditions, minimizing total daily cost and maximizing the use of cheap or surplus renewable electricity.

4.5 Electrical Devices for Water Heater Control and Power Measurement

To enable precise control and monitoring of the electric water heater, the Arduino UNO (*microcontroller unit - MCU*) interfaces with the heater through a combination of sensing and actuating circuits. These circuits are designed to ensure safe and accurate operation, allowing the Arduino to modulate precisely the heater’s power consumption based on the forecasting model’s predictions.

The **zero-crossing detector** (Figure 4) is a critical component for synchronizing the MCU’s control signals with the AC waveform. This circuit enables the MCU to determine the moments to trigger the power stage. The implementation uses an optocoupler to isolate the high-voltage AC signal from the low-voltage MCU circuit. The AC phase connected to the water heater is fed into the optocoupler’s input side, which consists of an LED. The LED illuminates whenever the AC voltage exceeds a certain thresh-

old, effectively converting the AC waveform into a pulsed signal. On the output side of the optocoupler, a phototransistor pulls the voltage at the MCU's input pin to ground when the LED is illuminated. A pull-up resistor connected to the MCU's input pin ensures a clean logic-high signal when the LED is off, creating a square wave that corresponds to the zero-crossings of the AC phase. This signal is then read by the MCU, allowing it to synchronize its control actions with the AC cycle.

The group discovered the hard way during a lab session that improperly sized resistors aren't particularly fond of too high powers — and they don't smell great when they decide to blow up.

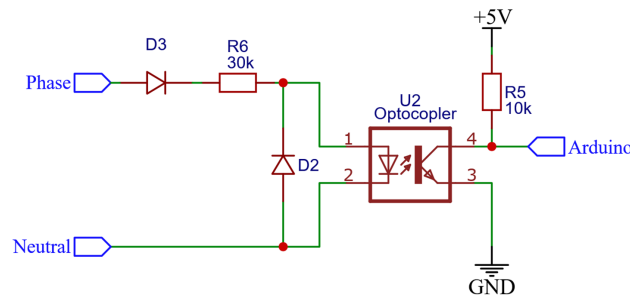


Figure 4: Schematic of the zero-crossing detection circuit using an optocoupler. The circuit isolates the AC phase from the MCU while providing a clean zero-crossing signal.

The **power stage** (Figure 5) is responsible for modulating the power delivered to the water heater based on the MCU's commands. This is achieved using a triac, which acts as a solid-state switch capable of handling the high currents required by the heater. The MCU controls the triac through an optotriac, which provides electrical isolation between the MCU and the high-voltage circuit. The control signal from the

MCU is first passed through a current-limiting resistor to the optotriac's input LED. When the LED is illuminated, the optotriac's output triggers the gate of the triac, allowing current to flow to the heater. The triac remains conductive until the next zero-crossing of the AC waveform, at which point it turns off unless re-triggered by the MCU. This approach, known as phase-angle control, enables the MCU to adjust the average power delivered to the heater by varying the conduction angle (*or delay*) of the triac.

A gate resistor is included in series with the triac's gate to limit the current and protect the optotriac. The entire power stage is connected in series with the water heater, ensuring that the MCU can directly control the heater's power consumption. This setup allows for fine-grained control over the heater's operation, enabling the system to respond dynamically to changes in energy availability or demand.

The group struggled with the power stage because we didn't realize that, despite being bidirectional, a triac still requires the gate current to come from one specific terminal [12]. All it took was swapping a single wire...

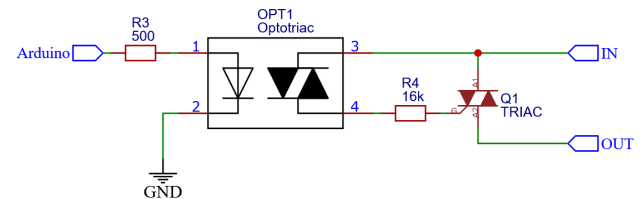


Figure 5: Schematic of the power stage circuit using an optotriac and a triac. The MCU controls the triac via the optotriac, enabling phase-angle control of the water heater.

The **current sensing** (Figure 6) circuit has yet not been implemented due to a lack of time. It should allow the Arduino to act in a closed-loop to adjust the delay and improve the precision of the whole device. The group would have used the ACS723 Hall-effect current sensor, which can provide real-time feedback on the water heater’s power consumption. This sensor would have been placed in series with the heater’s power line, enabling the MCU to measure the instantaneous current draw with high accuracy. The ACS723 outputs an analog voltage proportional to the current flowing through it, which can be then read by the MCU via an analog input.

The ACS723 can be powered by the MCU’s 5 V supply and is electrically isolated from the high-voltage AC circuit, ensuring safe operation. The sensor’s output can be filtered using a 10 nF capacitor to reduce noise and improve measurement stability. The MCU would then process the sensor’s output voltage, converting it into a current value using the sensor’s sensitivity (50 mV/A for the ACS723LLCTR-40AB-T model). This real-time current data can then be used to refine the power modulation strategy, enhancing the system’s responsiveness and accuracy. By integrating the ACS723 into the control loop, the system achieves precise energy management, optimizing the heater’s operation based on actual consumption rather than theoretical predictions.

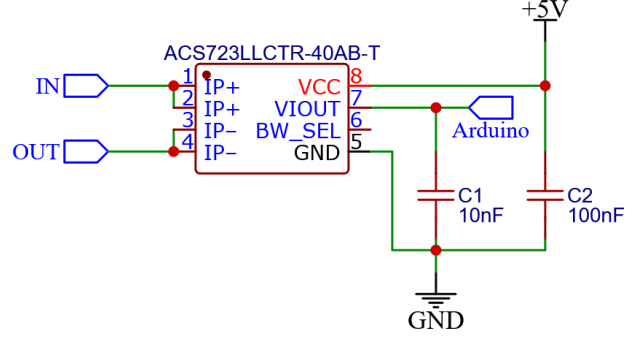


Figure 6: Schematic of the ACS723 current sensing circuit. The sensor would have been placed in series with the water heater, and its output connected to an Arduino analog input for real-time current measurement.

4.6 Water Heater Scheduling

The scheduling and activation of the electric water heater in this project is carried out using an LP-based optimization framework which integrates detailed forecasts of residential load, photovoltaic (PV) production, hot-water demand, and spot market electricity prices. The methodology begins with rigorous acquisition and pre-processing of all needed time-series inputs. Electrical load for the target household, a Swedish, fully electric residence with diverse appliances and dynamic occupant patterns, is forecasted using a machine learning approach that incorporates meteorological and historical data. PV generation is forecasted via the Forecast.Solar API, adjusted for local installation parameters and resampled into 15-minute energy intervals suitable for scheduling. Hot-water demand is measured and modeled from high-resolution pulse flow sensor data and converted to thermal energy requirements for the optimization model.

With the inputs prepared and resampled, the optimization model defines decision variables for each interval:

- Heater energy input
- Heater power setpoint
- Tank state of charge
- Solar and grid energy contributions
- A slack variable to capture any unmet hot water demand.

And the operational constraints are imposed based on:

- Equipment ratings
- Household comfort (minimum SoC / temperature)
- Solar excess availability
- Regulatory requirements such as tariff penalties or import limits

The objective function minimizes the total cost of grid-supplied energy, with a strong penalty for any use of the slack (fallback) variable, thereby ensuring comfort is prioritized but some emergency flexibility is allowed if needed.

The hot-water tank is modelled as a finite thermal storage with state of charge E_t [kWh] bounded between $E_{\min}$ and $E_{\max}$. These bounds are derived from the chosen tank parameters:

$$V = 360 \text{ L}, \quad \Delta T = 50^\circ \text{C}$$

$$c_p \approx 1.16 \times 10^{-3} \text{ kWh}/(\text{L} \cdot \text{K})$$

$$E_{\max} = V c_p (T_{\text{store}} - T_{\text{in}}) \quad (25)$$

$$E_{\min} = 0.1 \quad (26)$$

$$E_0 = 0.5 E_{\max} \quad (27)$$

A simple standing-loss factor (2%/h) and heater efficiency ($\eta_H = 0.98$) are included in the SOC update equation, so the water heater is never treated as an unlimited buffer and all comfort constraints on hot-water delivery are enforced through the bounds on E_t .

Once solved, the optimizer yields a schedule specifying the heater setpoint or input energy for each 15-minute slot. This schedule is then transformed into practical control commands for the water heater, typically a percentage of rated power or ON/OFF signal. For minute-level implementation, the interval schedule is further converted into 60-second modulation patterns using methods such as greedy assignment or waveform synthesis to match physical device capabilities. These commands are applied to the heater using an MCU (e.g., Arduino), which interfaces with the power stage (triac and optotriac) for phase-angle control and measures actual current via a Hall-effect sensor. Zero-crossing detection and safety interlocks ensure precise, low-noise switching and protect both the MCU and heater hardware.

Operationally, the system applies the scheduled setpoints, monitors tank temperature and system state, and periodically updates optimization inputs and solutions. If the model indicates that comfort requirements cannot be met (slack variable used), the controller automatically applies a fallback schedule for minimum service and

logs the event for diagnosis [13].

4.7 Real-time control application

The baseline water-heating schedule is generated offline by the optimisation model, which determines the minute-by-minute triac duty cycle required to minimise cost, maximise PV self-consumption, and satisfy comfort and technical constraints. To ensure that this schedule remains effective under real operating conditions, a real-time supervisory controller is implemented. This controller adapts the planned schedule based on live measurements of PV export, electricity prices, triac limits and an estimate of the water-tank state of charge (SOC). The SOC plays a key role, since it determines whether boosting or energy reallocation can be safely applied. Moreover, even if surplus PV power is available, the controller can only increase the boiler consumption up to the physical limit of the heating element ($P_{\max} = 2.2$ kW). This ensures that the real-time boosting never exceeds the maximum allowable triac output.

The real-time controller operates with a 10-second sampling period. At each step, it retrieves the current PV export power $P_{\text{export}}(t)$ from the NTMM data logger, together with an instantaneous estimate of the tank SOC. A hysteresis-based rule is used to identify periods of surplus solar energy. When the measured export exceeds a threshold (> 0.15 kW) and the triac can obtain a higher value, the controller enters a *boosting* mode, temporarily increasing the boiler power above the scheduled baseline to absorb the excess PV generation. Boosting is

deactivated when the export falls below a lower threshold (< 0.05 kW), preventing rapid switching.

Let $P_{\text{base}}(t)$ denote the scheduled power from the optimisation and $P_{\max} = 2.2$ kW the heating element rating. The commanded power is computed as:

$$P_{\text{cmd}}(t) = \begin{cases} P_{\text{base}}(t) + \min(A(t), B(t)), \\ P_{\text{base}}(t) \end{cases} \quad (28)$$

where $A(t)$ and $B(t)$ are defined as:

$$\begin{aligned} A(t) &= P_{\text{export}}(t), \\ B(t) &= P_{\max} - P_{\text{base}}(t) \end{aligned} \quad (29)$$

where the first case in (28) applies when boosting is active and the second case applies otherwise. After that it is converted into a triac duty ratio via:

$$d(t) = \min(P_{\text{cmd}}(t)/P_{\max}, 1) \quad (30)$$

To maintain long-term energy balance, the controller tracks both the *scheduled* cumulative energy $E_{\text{opt}}(t)$ and the *actual* delivered energy $E_{\text{act}}(t)$. Their difference,

$$E_{\text{credit}}(t) = E_{\text{act}}(t) - E_{\text{opt}}(t) \quad (31)$$

represents the additional energy delivered relative to the optimal plan, primarily caused by boosting. When $E_{\text{credit}}(t)$ becomes positive and exceeds a threshold, a simple reallocation mechanism is activated: future heating minutes with

the highest electricity prices are reduced or removed, provided that the tank SOC remains sufficient. In practice, the surplus energy absorbed during boosting is compensated by cutting an equivalent amount of scheduled heating from expensive periods. This ensures that the total daily heating requirement is maintained while shifting consumption away from high-cost hours.

In this implementation, the thermal and comfort constraints associated with the tank state of charge were handled as soft constraints through a placeholder SOC value. A real system would require direct communication with the tank's temperature sensor or a model-based SOC estimator to guarantee safe operation and fully enforce comfort limits.

Overall, the controller functions as an adaptive supervisory layer that refines the offline optimisation schedule based on real-world conditions. It aims to increase PV self-consumption, avoids non-profitable exports, and preserves cost optimality while dynamically responding to fluctuations in PV generation, household load, and electricity prices.

5 Application

This section presents the results of both the software implementation and the laboratory experiments. All software-based results correspond to the representative test day of 1 December 2025, which is used consistently throughout the input-generation and optimisation pipeline. First, the input signals and the optimisation-based output behaviour obtained from the software imple-

mentation that feeds the hardware are presented and analysed. Then, the corresponding laboratory testing results are reported, highlighting the practical considerations and the real-world behaviour of the system.

5.1 Software Implementation Results

The full input-generation pipeline (load forecasting, PV forecasting and boiler energy extraction), as well as the day-ahead optimisation, were executed on 30 November 2025 at 11:55. This produced the operating schedule for the next day (1 December 2025), which is analysed in the following figures. This workflow reflects the intended real-world deployment procedure, where the controller computes the heating plan one day in advance based on the available forecasts and market data.

5.1.1 Total load profile

By following the forecasting methodology described in subsection 4.1, the short-term domestic load predictions used by the heating optimisation algorithm are obtained. The resulting 24-hour load forecast and its cumulative energy trajectory for the representative day of 1/12/2025 are presented in Fig. 7 and Fig. 8, respectively.

5.1.2 Solar Forecasting

Next, applying the day-ahead solar forecasting method described in subsection 4.2, the expected PV production profile for the test day is computed. The corresponding day-ahead solar power

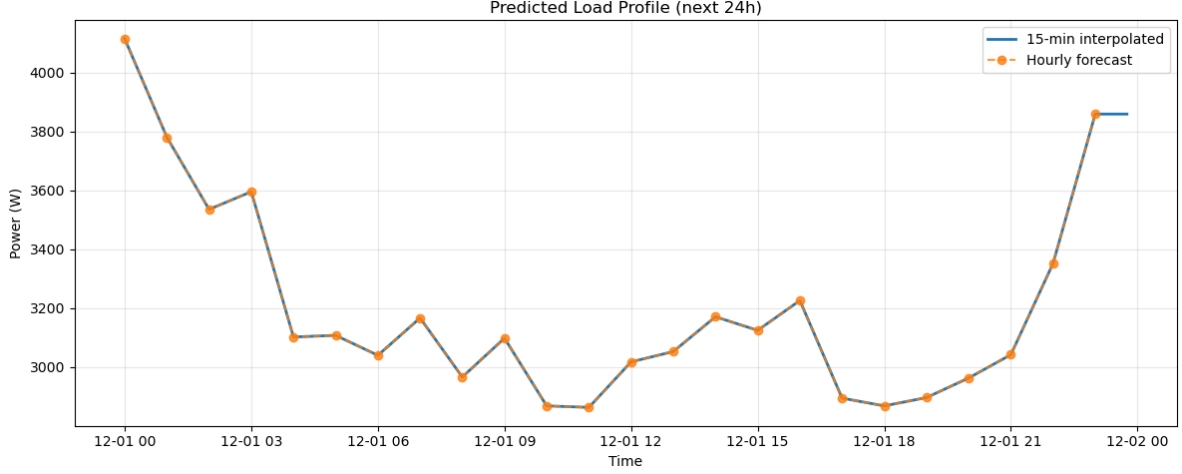


Figure 7: Predicted domestic load profile over the next 24 hours.

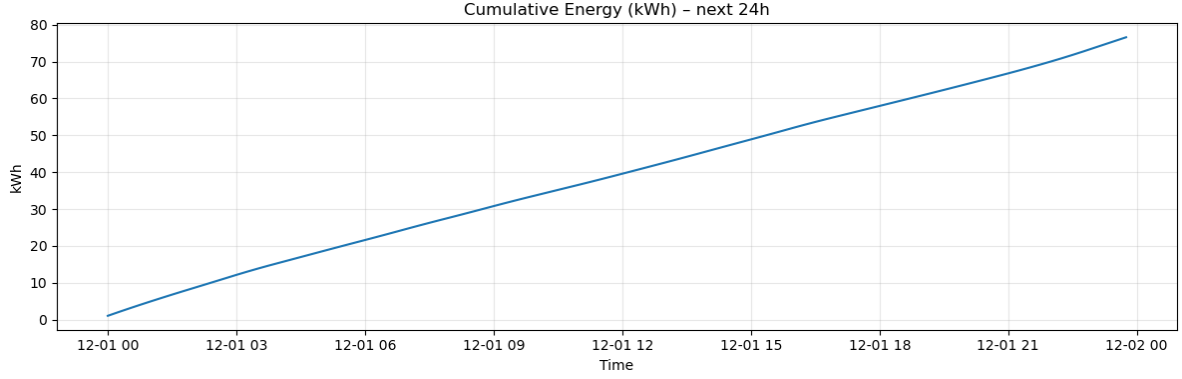


Figure 8: Cumulative predicted energy consumption (kWh) over the next 24 hours.

forecast is shown in Fig. 9 and serves as a key input to the optimisation model.

The solar forecasting process does not aim to be exact, but to quantify a potentially usable window of surplus solar energy. By intentionally committing a fraction of the boiler’s required energy to this surplus, the controller strategically maximizes self-consumption and reduces grid dependency, accepting short-term forecast uncertainty as part of the optimization process.

5.1.3 Boiler energy consumption profile

Following the procedure detailed in subsection 4.3, the boiler energy consumption profile for the selected day is extracted from the real hot-water usage data. This time series, presented in Fig. 10, represents a critical input to the optimisation, as it defines the total thermal energy demand that must be satisfied. Moreover, this time series clearly exhibits realistic daily usage patterns, with higher energy demand during peak hours and significantly lower consumption during the night and sleeping periods when hot-

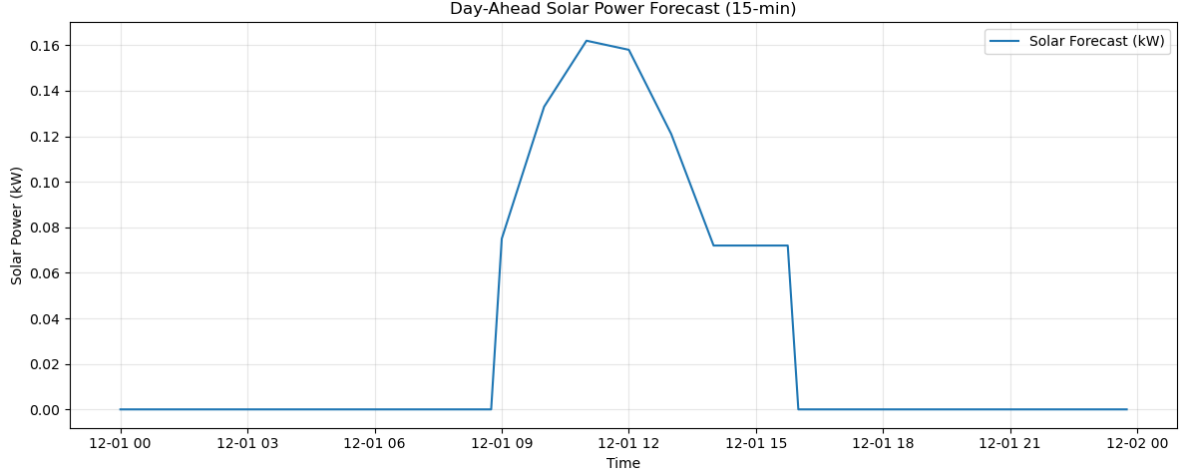


Figure 9: Day-ahead solar power forecast for the selected test day.

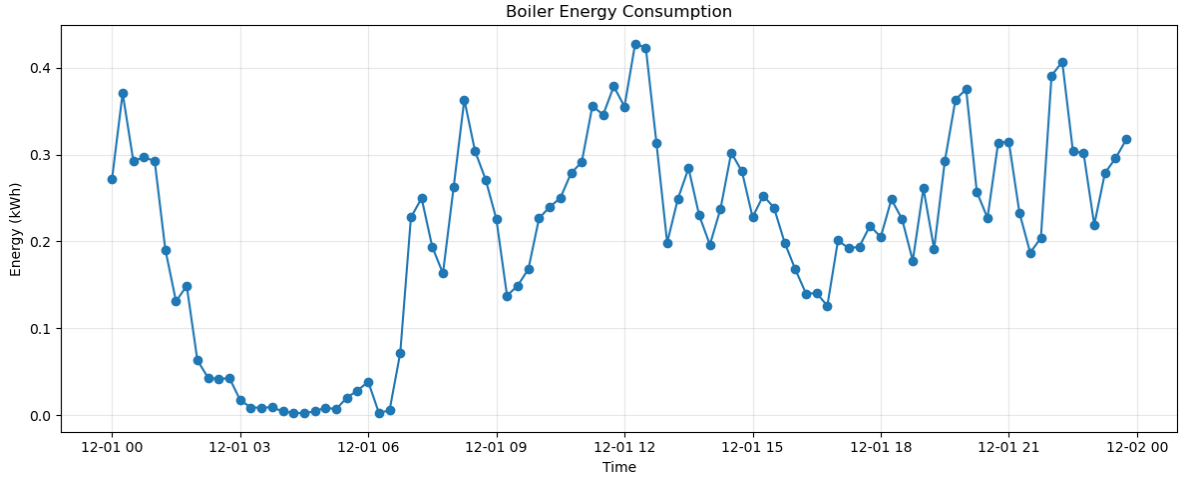


Figure 10: Expected boiler energy consumption for the selected day over the next 24 hours.

water usage is minimal. This behaviour aligns with a typical household load pattern and confirms that the processed water-heating data captures the underlying thermal dynamics of the domestic hot-water system.

5.1.4 Day-ahead electricity spot prices

The fetched day-ahead electricity spot prices, described in subsection 4.4, for the same testing date are shown in Fig. 11, providing the eco-

nommic signal that guides the optimisation toward low-cost intervals.

5.1.5 Inputs

Finally, to visualise the complete set of inputs used by the optimisation algorithm, Fig. 12 summarises the electricity price signal, the expected PV generation, and the net domestic load for the selected day. In this representation, the baseline household load has been adjusted by subtracting

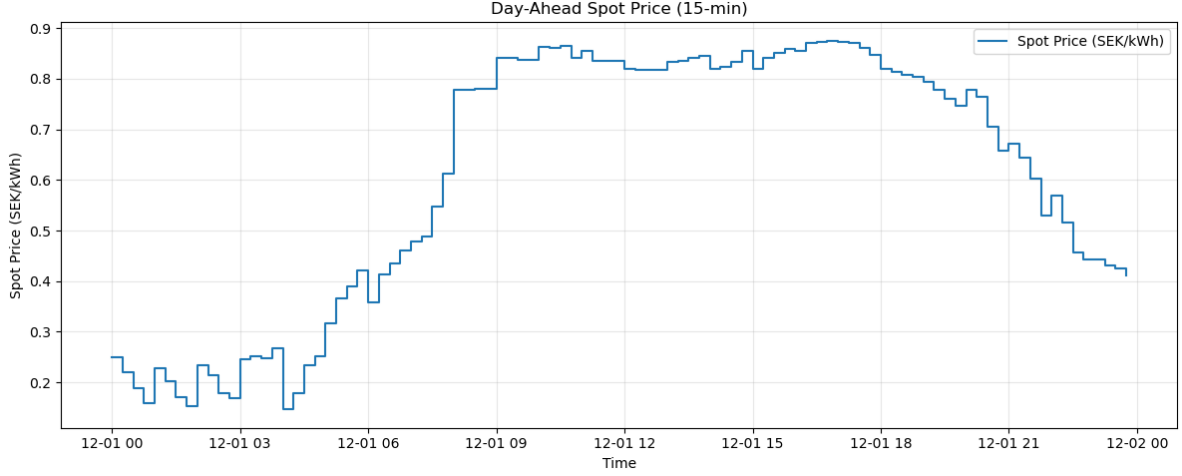


Figure 11: Day-ahead spot electricity prices.

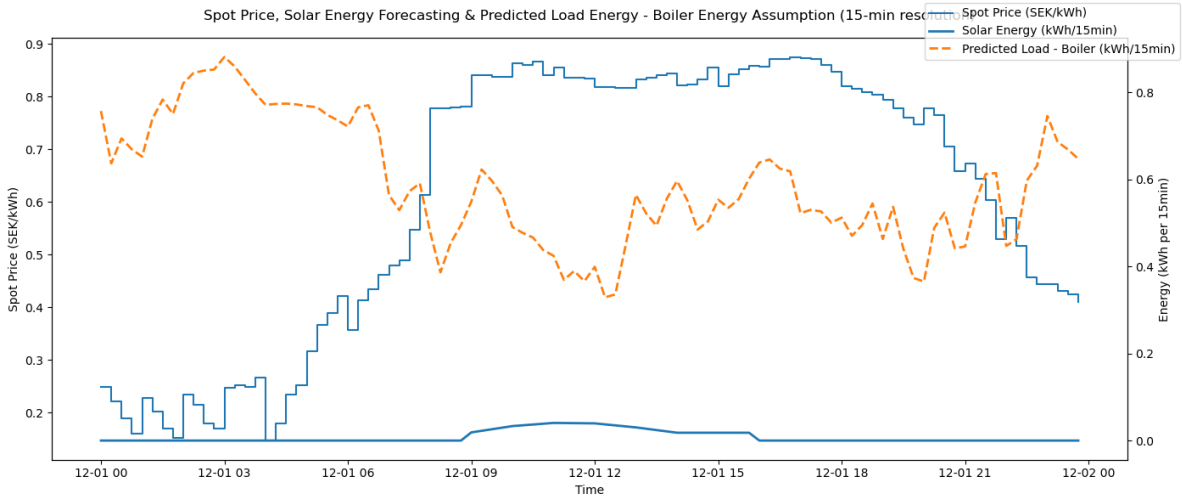


Figure 12: Spot prices, expected PV production and net domestic load without the boiler demand.

the hot-water boiler demand, so that the resulting *net load* reflects only the non-thermal portion of the consumption.

An important observation from this plot is that whenever the forecasted PV production (blue curve) rises above the net load (orange curve), the model anticipates a future surplus of solar energy. This expected surplus is automatically taken into account by the optimisation algorithm, which can shift part of the water-

heating demand into these intervals. As a result, the system is able to pre-heat the tank using low-cost or zero-cost PV energy, reducing grid consumption in later high-price periods.

5.2 Day-ahead optimised heating plan

Following the optimisation method described in subsection 4.6, this section presents the resulting next-day heating schedule for the representative winter test date of 1/12/2025. The un-

derlying forecasts (domestic hot-water demand, household electrical load and PV production) were generated on the previous evening. All plots use UTC timestamps, but they correspond directly to the analysed local day and the optimisation behaviour is unaffected.

Figure 13 summarises the 15-minute LP-based optimisation outcome. The top panel shows the tank state of charge remaining strictly within comfort bounds throughout the day. The optimiser pre-heats the tank during the low-price early-morning hours (00:00–06:00 UTC), reaching a maximum of approximately 17.6 kWh before allowing the level to decrease during periods of high demand and high prices. The minimum tank energy never drops below the 6 kWh comfort threshold, and the slack variable remains zero for all intervals, indicating that all hot-water demand is fully satisfied. The second panel shows the corresponding boiler power output, including the peak-avoidance mechanism. During the three highest-load intervals, the boiler is forced off (red markers), while during low-price hours it operates between 1.4–2.2 kW depending on the required charging rate. Moreover, the middle two panels decompose the energy flows: energy supplied to the boiler, actual hot-water demand, and the split between grid and PV contributions. As expected for winter conditions, PV excess is zero and the boiler relies almost on the grid. The optimiser nevertheless schedules heating strategically to allocate most charging to cheap-price periods. Finally, the bottom panel combines the dynamic spot price with the resulting operating cost per interval. The optimiser systematically avoids heating during the expen-

sive hours (07:00–16:00 UTC), instead shifting consumption into cheaper early-morning periods whenever sufficient storage is available. This demonstrates a clear alignment between thermal storage management and day-ahead price signals.

Figure 14 then illustrates how the optimised 15-minute boiler schedule is translated into minute-level control signals for the triac. The upper panel shows the resulting instantaneous boiler power, constrained by the 2.2 kW rating, while the lower panel shows the corresponding triac duty-cycle command between 0 (heater off) and 1 (full power). The resulting dispatch schedule can be seamlessly integrated with the model-adaptive real-time control layer, which updates the firing commands based on real-time PV generation and household export signals.

5.3 In-Laboratory Assembly and Testing

The prototype zero-crossing detector assembled on a small perfboard and mounted inside a plastic enclosure. Figure 15 shows the prototype implementation of the zero-crossing detector corresponding to the schematic in Figure 4 and the laboratory setup of bench testing.

The laboratory setup used to characterize the zero-crossing detector, including an 230 AC supply emulating the input stage and an oscilloscope monitoring the detector output and reference signals. During these initial tests, the circuit generate the clean, periodic square-wave expected as

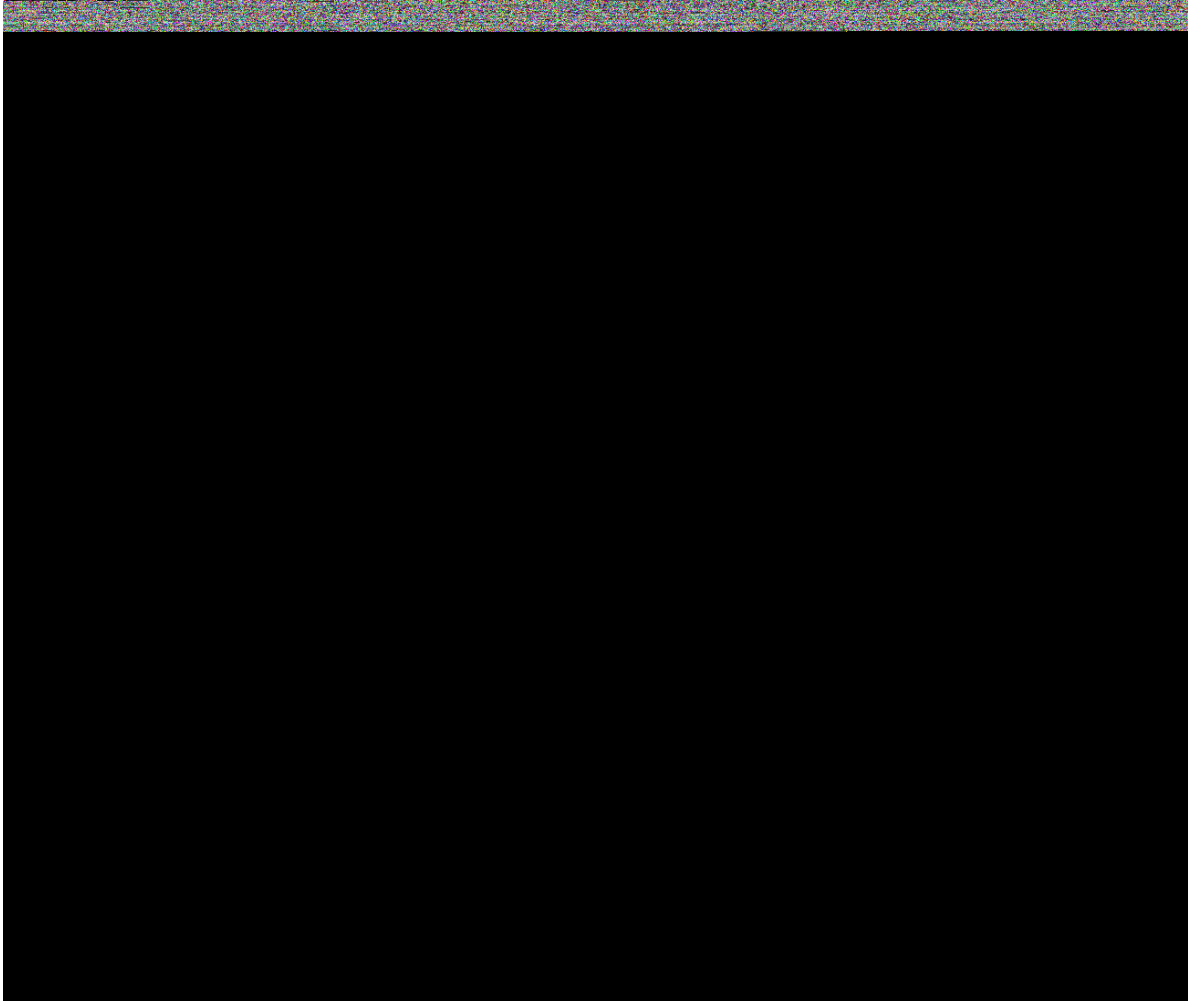


Figure 13: Example of next-day LP-based water heater optimisation showing over 24h (Time is in UTC).

one can see on Figure 16.

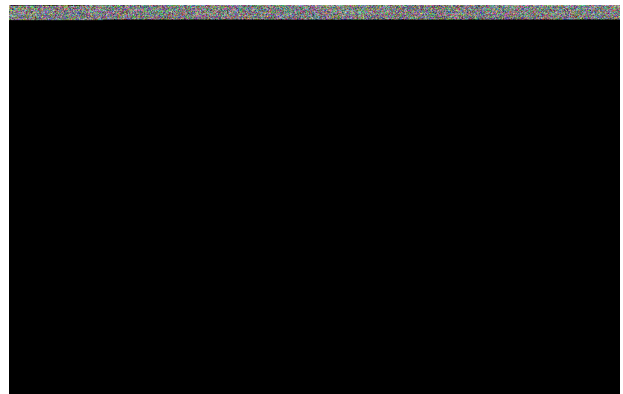


Figure 16: Oscilloscope picture showing the main sine voltage in purple (220V AC) and the output of the zero-crossing detector in yellow (5V square-wave)

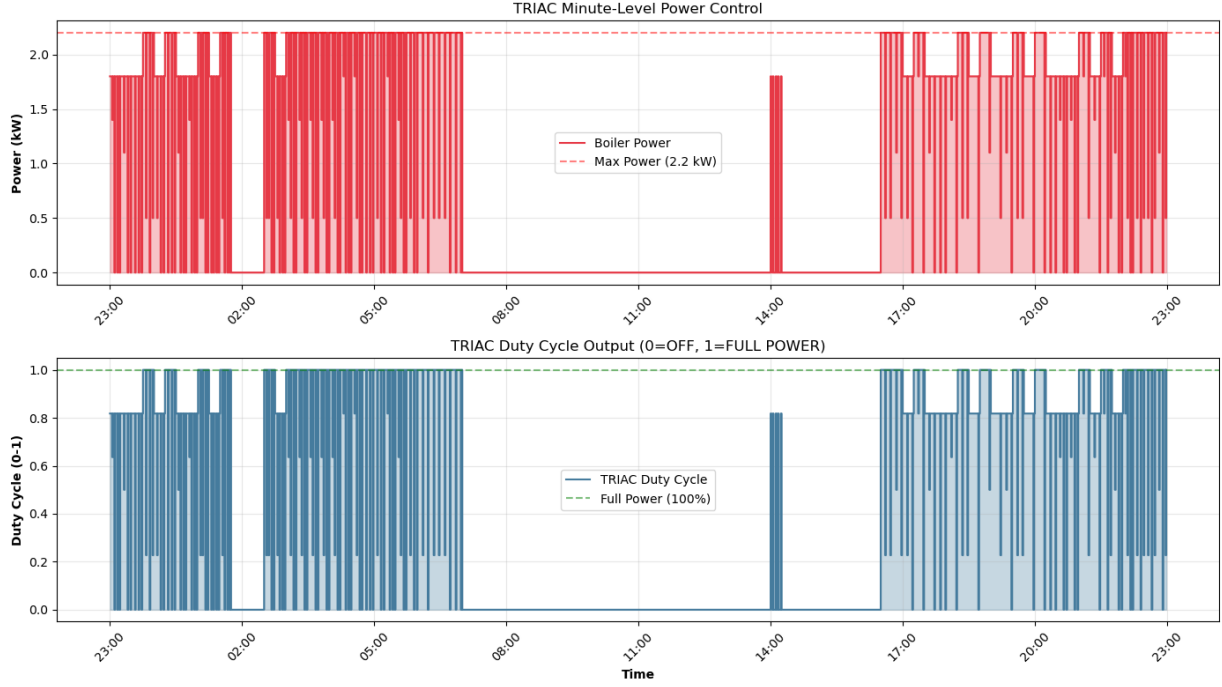


Figure 14: Minute-level implementation of the optimised boiler schedule, showing (top) the instantaneous heater power and (bottom) the corresponding triac duty-cycle required to realise the 15-minute LP setpoints.

The sensor is currently very accurate but dissipates a significant amount of energy in the input resistance. It might be worth investigating whether it would be possible to increase the value of the input resistance without losing too much accuracy.

The prototype power-stage was also assembled on a small perfboard and mounted inside a plastic enclosure. Figure 17 shows our supervisor, Nathaniel Taylor, double-checking our prototype implementation of the power-stage corresponding to the schematic in Figure 5 and the laboratory setup of bench testing.

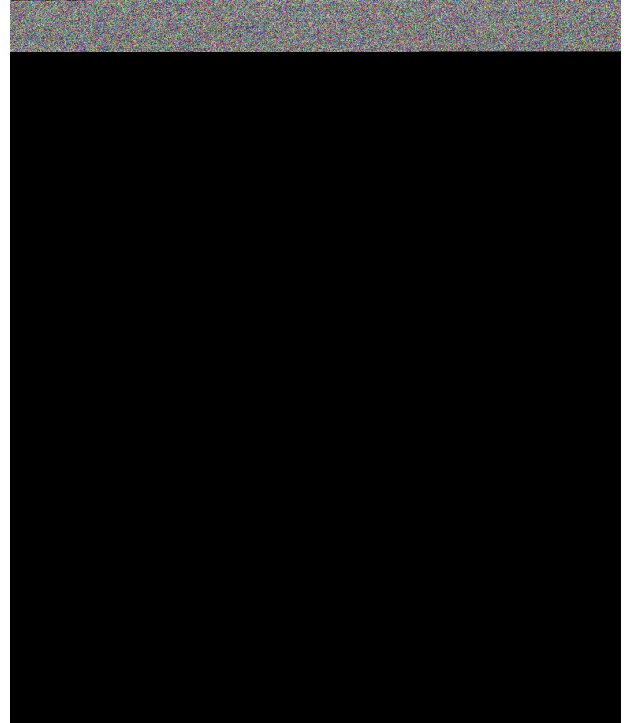


Figure 17: Supervisor Nathaniel Taylor inspecting the power-stage prototype.

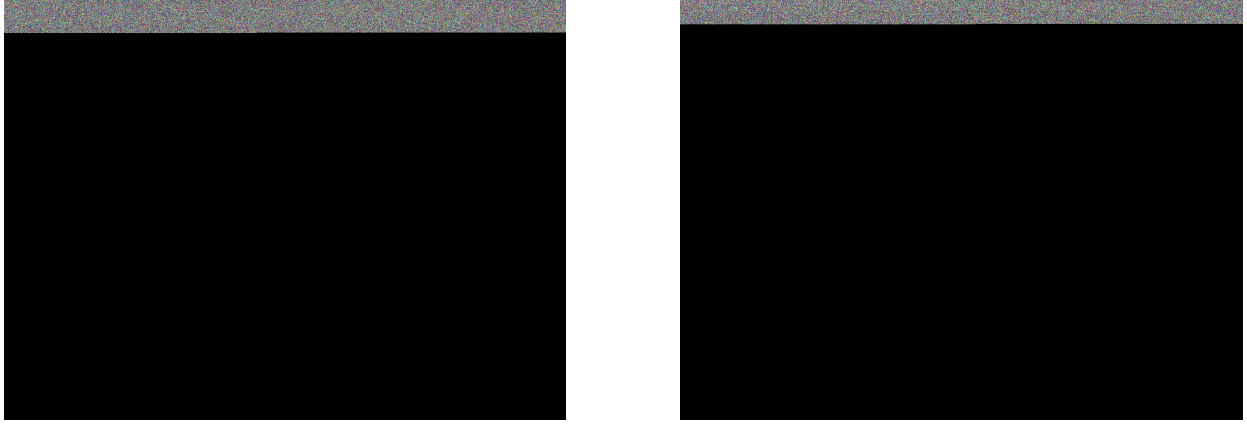


Figure 15: The Assembled Zero-crossing Detector Circuit (Left) and Laboratory Bench Testing (Right) of it.

As explained in Section 4.5, the group did not manage to find the time to make and test the current sensor.

5.4 Electronic implementation and testing

The practical implementation of the proposed smart control system involved the integration of both hardware and software components, each presenting unique challenges and learning opportunities. This section details the development process, the encountered issues, and the validation of the system’s core functionalities.

5.4.1 Hardware setup and initial testing

The system architecture relies on two main processing units: a **Raspberry Pi** for high-level data processing and optimization, and an **Arduino microcontroller** for real-time control and actuation. The initial phase focused on establishing communication between these components and validating their basic functionalities.

- **Raspberry Pi configuration:** As the team had no prior experience with Raspberry Pi, setting up the initial SSH connection and configuring the Python virtual environment posed initial challenges. However, these were resolved through systematic troubleshooting, ensuring a stable development environment for data acquisition, optimization, and communication with the Arduino.
- **Arduino initialization:** A basic script was implemented on the Arduino to blink an LED, confirming the functionality of the microcontroller and its programming environment. This step served as a foundational test for more complex tasks.
- **Communication protocol:** A Python script was developed on the Raspberry Pi to send ON/OFF commands to the Arduino via USB serial communication. The process involved:
 1. Connecting to the Raspberry Pi via SSH.

2. Executing a command in the SSH terminal to send instructions to the Arduino (connected via USB to the Raspberry Pi).
3. Validating the successful transmission and execution of commands on the Arduino by looking at the LED.

This test confirmed the robustness of the communication link between the two devices, a critical requirement for real-time control.

5.4.2 Algorithm implementation

The core optimization and real-time control algorithms were implemented on the Raspberry Pi. Minor challenges arose due to differences in library versions between the development environment and the Raspberry Pi. These issues were resolved by ensuring version compatibility, allowing the algorithms to run as intended.

To validate the control logic, an Arduino script was developed to interpret the algorithm commands and apply them to an LED, simulating the behavior of the water heater. This stage was crucial because it drew on all the work that had been done beforehand. Its success was therefore very important and eagerly expected.

5.4.3 Final global tests

Now that every single module is working, and that the communication between the Arduino and the RaspberryPi is stable, the final step is to test everything together. Figure 18 shows the

overall test bench with everything connected together. Figure 19 shows the global results of our project.

The group finished the lab sessions proudly watching the light change intensity following the algorithm’s decisions, all in complete autonomy.



Figure 19: Picture of the oscilloscope during the final test showing the output of the zero-crossing detector in blue (square wave), the Arduino trigger to the power stage in yellow (pulse), and the output signal feeding the bulb (truncated sine wave).

6 Limitations of the Study

Despite demonstrating the feasibility of combining day-ahead optimisation with real-time control and hardware actuation, several limitations remain in the present work. First, the system has not yet been validated on a real water heater. All results involving thermal behaviour, comfort constraints, and energy balancing rely on models and offline simulations. Since the prototype lacks a temperature sensor interface or a complete state-of-charge estimator, the real-



Figure 18: The overall test bench with the Arduino (bottom), the Raspberry Pi (black box in the middle), the zero-crossing detector (under the Raspberry Pi), the power stage (right), the load (light bulb on), the Raspberry Pi monitoring screen (bottom right) and the oscilloscope (top).

time controller operates using placeholder SOC values. This prevents a fully reliable enforcement of comfort and safety constraints in practical operation.

Second, the hardware implementation, while functional, is incomplete. The current-sensing stage was not assembled due to time constraints, which means that the controller cannot close the loop with real power measurements. As a result, the triac firing control operates in open loop and assumes a linear relationship between delay and delivered power, an approximation that may not hold under distorted mains waveforms or varying load conditions. Similarly, the zero-crossing

detector prototype, although working, dissipates more energy than expected and may require redesign to ensure long-term robustness.

Third, the optimisation framework depends strongly on forecast accuracy. Load, PV production, and hot-water consumption forecasts are all subject to uncertainty. Although the real-time controller partially compensates for forecast errors through boosting and reallocation mechanisms, a sustained mismatch—such as prolonged overcast conditions or atypical user behaviour—could reduce the system’s economic performance or challenge its ability to maintain comfort.

Finally, the laboratory tests were limited in scope. They were conducted on a resistive light bulb rather than on a high-power heating element, and the system was not tested over extended periods or under diverse operating conditions. As a result, long-term reliability, thermal dynamics, and real-world interactions between optimisation and hardware remain to be evaluated.

7 Conclusion

This project demonstrates that an LP-based controller, combined with a physically constrained hot-water tank model, can effectively shift boiler operation toward periods of low electricity prices and high photovoltaic (PV) availability, while maintaining the state of charge within predefined comfort limits. The generated day-ahead schedules and the corresponding minute-level triac duty-cycle commands show that the optimisation outputs can be translated into feasible actuation signals for a resistive heater.

In the laboratory, the core hardware components—Arduino microcontroller, triac power

stage, and zero-crossing detector—were successfully assembled and tested, confirming the correct triggering behaviour and validating the essential building blocks of the control chain. These experiments provide a strong foundation for future integration, even though full real-world validation on an actual water heater remains to be carried out.

The next steps include implementing accurate current feedback, integrating a reliable temperature or state-of-charge estimation method, and conducting extended measurement campaigns. These improvements will enable a closed-loop implementation, allowing for more precise power modulation, enhanced safety, and a robust assessment of real-world performance. Future work should also investigate the long-term impacts on cost savings, PV self-consumption rates, and peak-load reductions under diverse weather conditions and household behaviours.

Overall, this project validates the technical feasibility of a smart, model-driven water-heater controller and establishes a clear pathway toward a fully operational, field-ready system capable of supporting residential flexibility and improved self-consumption in modern energy systems.

8 Reference List

References

- [1] Nord Pool AS, *Day-Ahead Auction Prices, Delivery Area SE3*, <https://data.nordpoolgroup.com/auction/day-ahead/prices?deliveryDate=latest¤cy=EUR&aggregation=DeliveryPeriod&deliveryAreas=SE3>, 2025.

- [2] International Energy Agency, *Sweden: Tax reduction for micro-scale electricity production*, 2025.
- [3] Ellevio AB, *Elnätspriser för privatkunder*, <https://www.ellevio.se/abonnemang/elnavtspriser-privat/>, 2025.
- [4] Svenska kraftnät, *National Grid – Tariff and Charges*, <https://www.svk.se/en/national-grid/>, 2025.
- [5] L. Lundström, *A milp-based energy management system*, DiVA Portal, 2021.
- [6] A. Sahin, *Residential water heater optimization with time-of-use tariffs and pv self-consumption*, Energy, 2021.
- [7] Y. Wang, *Comprehensive framework for smart residential demand-side management*, Nature, 2025.
- [8] Swedish Meteorological and Hydrological Institute, <https://www.smhi.se/>, 2025.
- [9] Elpriset Just Nu, *Elpriset Just Nu – Swedish Electricity Spot Prices*, <https://www.elprisetjustnu.se>, 2025.
- [10] *House, electricity meter and water meters information*, <https://ntmm.org/pqlog/>, 2025.
- [11] Forecast.Solar, *Forecast.solar api documentation*, <https://forecast.solar/>, 2025.
- [12] STMicroelectronics, *Application note : Controlling a Triac with a phototriac*, https://www.st.com/resource/en/application_note/dm00451014-controlling-a-triac-with-a-phototriac-stmicroelectronics.pdf, July 2020.
- [13] S. Mhanna, *Optimal management of storage-based flexible loads in smart grids*, Electric Power Systems Research, 2023.

GitHub Repository – Multicriteria Load Control